Programación Concurrente y Paralela

Paralelismo

Fork-join

Multiplicar matrices en paralelo

El problema

calcular $C \in \mathbb{R}^{n \times n}$; $C = A \cdot B$, i.e., $c_{ij} = \sum_{k=0}^{n-1} a_{ik} \cdot b_{kj}$.

- Idea "directa": **un thread por cada entrada** c_{ij}, es decir, n^2 threads.
- Cada thread calcula un producto escalar y termina. El programa los arranca con `start()` y los espera.

¿Es un buen diseño?

Es altamente paralelo y los threads no necesitan sincronizarse entre sí, pero la eficiencia depende de la plataforma.

Costo de creación de threads

- Cada thread ocupa **memoria** (*stack*, bookkeeping) y crearlo, planificarlo y destruirlo cuesta cómputo.
- Si bien plataformas actuales ahora permiten la creación de threads virtuales (Go, Java con Project Loom), la creación de threads *de sistema operativo* es ineficiente.

Alternativa: pool de threads

Threads **de larga vida**: cada worker espera a que le asignen una *tarea* (unidad corta de trabajo), la ejecuta, y busca la próxima.

- Bonus: el pool **aísla** al programador de la plataforma. La cantidad total de threads es configurable y se puede calibrar para correr bien en un procesador, en una laptop o en un multiprocesador grande.

ExecutorService

- Java tiene un mecanismo de manejo de pool de threads que abstrae en la interfaz `ExecutorService`.
- De sus implementaciones, `ThreadPoolExecutor` es la más directa, que usa una única cola de tareas, generando contención.
- Por otro lado, `ForkJoinPool` usa una cola para cada thread `Worker` y les permite hacer *work-stealing* si uno se queda sin tareas.
- En `ForkJoinPool` se usa `fork()` y `join()` con `RecursiveTask<T>` y `RecursiveAction`.

Referencia de Fork-Join

`ForkJoinPool` y tareas fork-join

```
ForkJoinPool forkJoinPool = new ForkJoinPool();

// Tareas que devuelven un valor: RecursiveTask<T>
// Tareas con efectos:            RecursiveAction
//   task.fork()    correr la tarea (eventualmente) en el pool
//   task.join()    espera a que termine y da el resultado
//                  *(el worker puede tomar otra tarea mientras tanto)*
//   task.compute() hace el trabajo en el propio worker
```

Ejemplo 0: suma de matrices en fork-join

- Tomamos una clase `Matrix` que nos permite hacer `get()`, `set()`, y `split()` (dejándonos una nueva matriz pero que apunta al mismo espacio de memoria)
- Suponiendo por simplicidad dimensión potencia de 2, hacemos la tarea diviendo las matrices sumandos en 4 cuadrantes, hasta llegar a un umbral mínimo (por ejemplo, matriz de 2×2)
- Usamos `fork()` para lanzar una tarea por cada cuadrante; luego esperamos que todos hayan sido calculados con `join()` (siempre queremos demorar los `join()` lo más posible).
- Como las tareas no tienen un valor de retorno (escriben una matriz de salida), se implementan como `RecursiveActions`.

Suma de matrices en fork-join, código

```
public class MatrixAddTask
    extends RecursiveAction {
  static final int THRESHOLD = ...; //umbral para hacer la suma a mano

  public void compute() {
    if (lhs.getDim() <= THRESHOLD) {
      Matrix.add(lhs, rhs, sum);
    } else {
      List<MatrixAddTask> tasks =
          new ArrayList<>(4);
      for (int i = 0; i < 2; i++)
        for (int j = 0; j < 2; j++)
          tasks.add(new MatrixAddTask(
              lhs.split(i, j),
              rhs.split(i, j),
              sum.split(i, j)));
      tasks.forEach(task -> task.fork());
      tasks.forEach(task -> task.join());
    }
  }
}
```

Ejemplo 1: producto de matrices

Para el producto, un cuadrante de la matriz resultado es más complicado que el de las matrices de origen. Si $C = A \cdot B$, usamos la identidad

$$\begin{aligned}\begin{pmatrix} C_{00} & C_{01} \\ C_{10} & C_{11} \end{pmatrix} &= \begin{pmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{pmatrix} \cdot \begin{pmatrix} B_{00} & B_{01} \\ B_{10} & B_{11} \end{pmatrix} \\ &= \begin{pmatrix} A_{00} \cdot B_{00} & A_{00} \cdot B_{01} \\ A_{10} \cdot B_{00} & A_{10} \cdot B_{01} \end{pmatrix} + \begin{pmatrix} A_{01} \cdot B_{10} & A_{01} \cdot B_{11} \\ A_{11} \cdot B_{10} & A_{11} \cdot B_{11} \end{pmatrix}.\end{aligned}$$

Y aprovechamos que tenemos la suma paralelizada.

Producto de matrices en fork-join, código

```
public class MatrixMulTask
    extends RecursiveAction {
  static final int THRESHOLD = ...; //umbral para hacer el producto a mano

  public void compute() {
    int n = lhs.getDim();
    if (n <= THRESHOLD) {
      Matrix.multiply(lhs, rhs, product);
    } else {
      List<MatrixMulTask> tasks = new ArrayList<>(8);
      Matrix[] term = new Matrix[]{new Matrix(n), new Matrix(n)};
      for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
          for (int k = 0; k < 2; k++) {
            tasks.add(
              new MatrixMulTask(lhs.split(j, i), rhs.split(i, k),
                term[i].split(j, k)));
          }
        }
      }
      tasks.stream().forEach((task) -> {task.fork();});
      tasks.stream().forEach((task) -> {task.join();});
      (new MatrixAddTask(term[0], term[1], product)).compute();
    }
  }
}
```

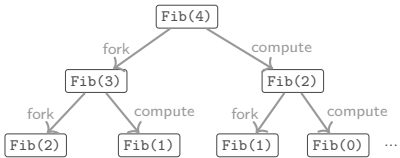
Ejemplo 2: Fibonacci

- Usamos como ejemplo esta recursión clásica.
- Usar paralelismo para esta tarea es algo ineficiente, pero nos sirve para entender el concepto.
- En vez de `RecursiveActions` usamos `RecursiveTasks` (ya que queremos un valor de retorno).
- Idea: Hacer `fork()` un paso recursivo *y ejecutar el otro como parte de la tarea actual* (con `compute()`).
- Luego del `compute()`, se hace `join()` para capturar el resultado de la llamada recursiva lanzada como tarea.

Fibonacci: Código

```
class FibTask
    extends RecursiveTask<Integer> {
  int arg;

  protected Integer compute() {
    if (arg > 1) {
      FibTask right = new FibTask(arg-1);
      right.fork();
      FibTask left = new FibTask(arg-2);
      return left.compute() + right.join();
    } else {
      return arg;
    }
  }
}
```



Nota 1: En el libro hace primero el `join()`, a pesar de explicar lo contrario. **Nota 2:** El libro llama *right* a la rama del árbol que se dibuja a la izquierda y *left* a la que se dibuja a la derecha. Se dejó como estaba.

Dependencia de pasos

- Tomemos nuestro cómputo y dividámoslo en pasos dentro de cada tarea.
- En este análisis nos interesa la capacidad de paralelización; nos abstraemos de las instrucciones y sólo distinguimos pasos *predecesores* y *sucesores* en términos de `fork()`, `join()` y `compute()`.

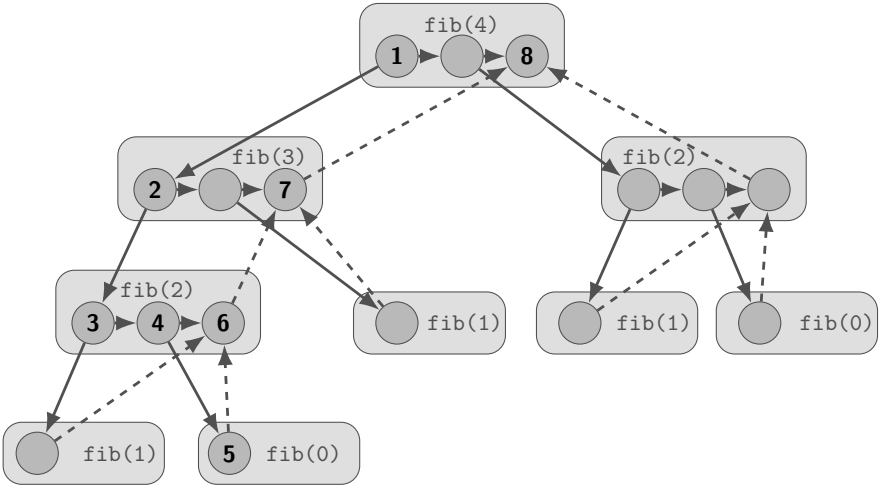
DAG de cómputo: definición

- Vemos el cómputo como un **grafo acíclico dirigido** (DAG) donde cada nodo es un paso.
- Un paso con una invocación a `fork()` o a `compute()` tiene dos aristas de salida, el paso siguiente de la tarea y el primero de la tarea invocada.
- Si el paso tenía un `compute()`, entonces hay otra arista que va del paso final de la tarea invocada al *siguiente* del que hizo el `compute()` que está en la misma tarea.
- Si en una tarea t hay un `fork()` a una tarea t', entonces en t existe un paso con un `join()` de t'. El paso final t' tiene una arista que va al nodo con ese `join()`.

DAG de cómputo: camino crítico

- Podemos ver que el digrafo resultante es acíclico, las aristas siempre van en dirección de avance en el tiempo.
- El elemento que nos interesa de este grafo es su *camino más largo* comenzando desde el paso inicial de la tarea inicial.
- Este camino es el *camino crítico*, la secuencia de pasos interdependientes que da una cota mínima para la ejecución total del cómputo.

DAG de cómputo para Fibonacci



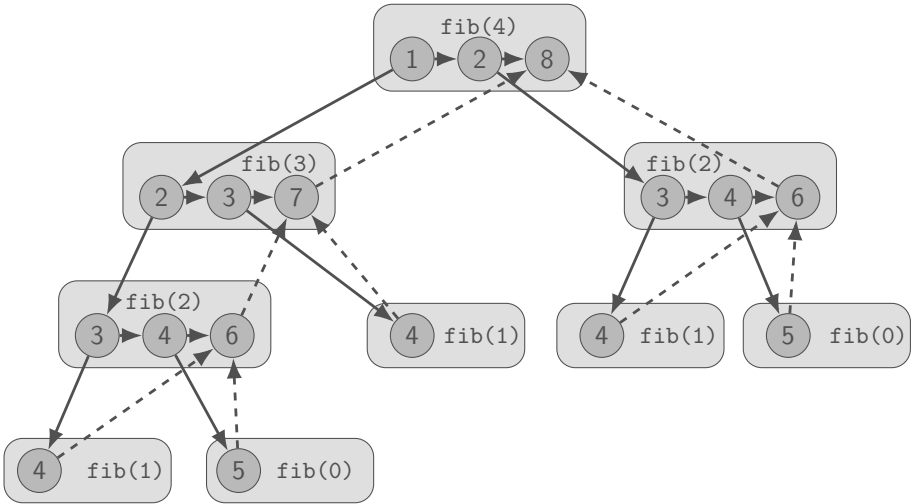
Las aristas rayadas son resultantes de las acciones `join()`. Se marcan los 8 nodos del camino crítico.

Métricas de cómputo paralelo

La *latencia* T_P de un programa multithread es el tiempo mínimo (en pasos) que toma ejecutarlo usando P procesadores:

- *trabajo* (*work*): T_1, el tiempo en pasos que toma el cálculo ejecutado en un solo procesador (i.e., el total de pasos de todo el programa).
- *duración* (*span*): T_∞: el tiempo en pasos que toma el cálculo ejecutado en una cantidad no acotada de procesadores.
- Notar que el *span* es la longitud del camino crítico: como ningún paso se queda sin procesador, todo paso ejecuta apenas terminan sus predecesores, así que la cantidad de pasos total está dada por el camino más largo.
- En el ejemplo de Fibonacci, se encuentra marcado el camino que determina el *span*, de longitud 8.

Fibonacci, paso de ejecución de las tareas en T_∞



El número en los nodos es el orden de ejecución (con paralelismo) de cada paso.

Cotas de la latencia

- $T_P \geq T_1/P$: ley del *work*, no es posible terminar más rápido que repartir los pasos perfectamente entre los P procesadores.
- $T_P \geq T_\infty$: ley del *span*, no es posible ganarle a la ejecución *span* sin romper interdependencias entre pasos.
- Llamamos *aceleración* (*speedup*) a T_1/T_P; un programa donde se logra speedup lineal es tal que $T_1/T_P \in \Theta(P)$.
- Llamamos *paralelismo* (*parallelism*) a T_1/T_∞, es el speedup máximo alcanzable.
- El paralelismo es una métrica de la cantidad de tareas disponibles en promedio, lo que ayuda a medir el valor óptimo de P.

Análisis de los ejemplos: Suma

Sea $A_P(n)$ el tiempo que toma sumar dos matrices de dimensión n con P procesadores.

$A_1(n)$ es una recursión que hace 4 sumas de matrices de tamaño $n/2$ más $\Theta(1)$ de trabajo extra:

- $A_1(n) = 4\,A_1(n/2) + \Theta(1) = \Theta(n^2)$
- $A_\infty(n) = A_\infty(n/2) + \Theta(1) = \Theta(\log n)$ (en el *span*, la ejecución de las 4 tareas se da en paralelo)
- Paralelismo: $\Theta(n^2/\log n)$

Análisis de los ejemplos: Producto

Sea $M_P(n)$ el tiempo que toma multiplicar dos matrices de dimensión n con P procesadores. Recordamos la cuenta que usamos:

$$\begin{pmatrix} C_{00} & C_{01} \\ C_{10} & C_{11} \end{pmatrix} = \begin{pmatrix} A_{00} \cdot B_{00} & A_{00} \cdot B_{01} \\ A_{10} \cdot B_{00} & A_{10} \cdot B_{01} \end{pmatrix} + \begin{pmatrix} A_{01} \cdot B_{10} & A_{01} \cdot B_{11} \\ A_{11} \cdot B_{10} & A_{11} \cdot B_{11} \end{pmatrix}.$$

$M_1(n)$ es una recursión que hace 8 productos de matrices de tamaño $n/2$ y una suma de matrices de tamaño n.

- $M_1(n) = 8\,M_1(n/2) + A_1(n) = 8\,M_1(n/2) + \Theta(n^2) = \Theta(n^3)$
- $M_\infty(n) = M_\infty(n/2) + A_\infty(n) = M_\infty(n/2) + \Theta(\log n) = \Theta(\log^2 n)$
- Paralelismo: $\Theta(n^3/\log^2 n)$ - En matrices de dimensión 1000 se pueden aprovechar 10^7 procesadores.

Estimando T_P

El análisis que hacemos es teórico y está fuertemente idealizado

- Asume que siempre hay un procesador donde los threads puedan correr
- Asume que los procesadores están dedicados, no hay otros procesos y la cantidad disponible no varía.
- Tenemos que considerar que tenemos **tareas**, que tienen que ser tomadas por **threads**, que tienen que tener asignado un **procesador**.

Scheduler idealizado

El análisis que hacemos es teórico y está fuertemente idealizado

- Consideremos, para nuestra estimación, que necesitamos un paso de ejecución en un procesador para completar un paso del DAG de cómputo.
- Más aún, supongamos que trabajamos con un scheduler *greedy*, es decir, que en cada paso de ejecución toma de todos los pasos listos para ejecutar de nuestro DAG posibles (a lo sumo P de ellos) y los ejecuta en los procesadores (el paso del DAG está listo cuando sus predecesores ya fueron computados).
- Ojo, la manera de elegir **no da igual**, pero vamos a enunciar teoremas sobre cualquier manera de elegir.

Teoremas de scheduling *greedy*: cota

Teorema

Dado un programa multithread con *work* T_1 y *span* T_∞, y P procesadores, ejecutar el programa con un scheduler *greedy* toma tiempo T_P^* donde:

$$T_P^* \leq \frac{T_1}{P} + T_\infty$$

Idea de demostración: Categorizamos los pasos de ejecución en *completos*, si tengo P pasos del DAG para asignar, o *incompletos*, si tengo menos de P. La cantidad de pasos de ejecución completos no puede ser mayor a T_1/P (corresponde a dividir la tarea perfectamente en P), y la cantidad de pasos incompletos no puede ser mayor a T_∞ (en cada paso donde uso menos de P procesadores seguro estoy tomando el primer nodo de cada camino crítico, que está listo (si no el camino crítico sería más largo); luego, al ejecutar el paso estoy reduciendo el *span*).

Teoremas de scheduling *greedy*: aproximación

Teorema

El tiempo T_P^* es a lo sumo el doble de tiempo que el scheduling óptimo (T_P).

Demostración: Usando las cotas de la ley del *work* y la ley del *span* juntas:

$$T_P \geq \text{máx}(\frac{T_1}{P}, T_\infty)$$

mientras que por el teorema anterior

$$T_P^* \leq \frac{T_1}{P} + T_\infty \leq 2\,\text{máx}(\frac{T_1}{P}, T_\infty)$$

luego,

$$T_P^* \leq 2\,T_P$$

□

Suboptimalidad de un scheduler *greedy*

- El teorema anterior dice que un scheduler greedy genérico toma a lo sumo el doble de pasos que la estrategia óptima.
- Remarcamos que la estrategia óptima *también está* entre las greedy, es la que hace la elección más eficiente de los pasos a ejecutar.
- Ejemplo: Tenemos $P = 2$ y 4 pasos a ejecutar: A_1, su sucesor A_2, B y C (que son independientes entre sí y de los pasos A).
- El scheduler genérico podría elegir hacer $\{B, C\}$ en paralelo primero, luego $\{A_1\}$, pero está forzado a hacer $\{A_2\}$ en el paso siguiente.
- Sin embargo, le gana hacer $\{A_1, B\}$, $\{A_2, C\}$.
- Este scheduler greedy necesitó 3 pasos de ejecución contra los 2 del óptimo; lo que sabemos es que no hay elección que use más de 4 pasos de ejecución.

Work stealing

Distribución de trabajo

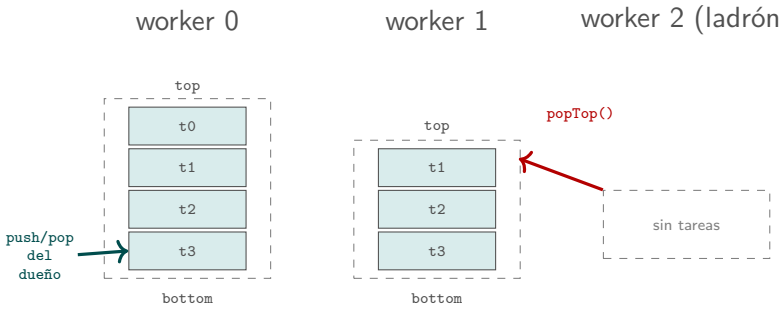
Nosotros generamos tareas y queremos que los threads las puedan tomar para trabajar con ellas, ¿Cómo balanceamos la carga?

- *work dealing*: los workers sobrecargados asignan tareas a los workers que tienen menos carga (pero, ¿cómo elige cuándo y a quién dársela?).
- *work stealing*: cada worker le *roba* trabajo a otro cuando se queda sin tareas. La elección de a quién sacarle es más sencilla.

Pool con *work-stealing*

- Cada worker tiene su propia cola de tareas local, almacenada en una *double ended queue* (*deque*).
- De un lado de la deque (usamos el `Bottom`), hay métodos `pushBottom()` y `popBottom()`. Las tareas ingresan por medio del `pushBottom()` y son tomadas por el worker con `popBottom()`, en orden LIFO (nunca se encola desde el `Top`).
- Al inicio, todas las deques están vacías, y a un worker se le da explícitamente la tarea inicial del problema.
- Cuando un worker tiene que hacer `fork()`, este crea una tarea nueva la agrega a *su propio deque* con `pushBottom()`.
- Un worker con su deque de tareas vacía se vuelve *ladrón* y *roba* la tarea más vieja (obtenida con `popTop()`) del deque de otro worker, su *víctima*.

Work stealing: esquema



- El dueño hace `pushBottom()` y `popBottom()`: tareas *nuevas*, posiblemente calientes en la caché.
- El ladrón roba con `popTop()`: tareas *viejas*.
- Cada uno trabaja en un extremo distinto, generando menos contención.

Funcionamiento del robo de tareas

- la elección de la víctima es aleatoria. Si la víctima elegida no tiene tareas, prueba con otra.
- Antes de concretar el robo, el ladrón hace `Thread.yield()`, es decir, cede su procesador. La idea es dárselo a un thread que no tenga procesador pero sí una tarea activa.

Un pool con *work stealing*

```
public void run() {
  int me = ThreadID.get();
  RecursiveAction task = queue[me].popBottom();
  while (true) {
    while (task != null) {                    trabajo propio
      task.compute();
      task = queue[me].popBottom();
    }
    while (task == null) {                    trabajo ajeno
      Thread.yield();
      int victim = ThreadLocalRandom.current().nextInt(queue.length);
      if (!queue[victim].isEmpty())
        task = queue[victim].popTop();
    }
  }
}
```

Nota: Este worker sigue intentando tomar tareas por siempre. Si queremos que eventualmente termine, podemos utilizar una *barrera de terminación*.

Invariante de `popTop()`

- Idealmente, queremos una deque linealizable; todo `popTop()` devuelve una tarea si hay alguna.
- En la práctica, `popTop()` puede devolver `null` si compite con otro ladrón; en ese caso el perdedor no espera, reintenta con otra víctima.

Idea de implementación

- Por diseño, sólo el thread **dueño** de una deque hace `pushBottom()` y `popBottom()` de la misma, luego no hace falta implementar mecanismos de contención.
- Los **ladrones**, en cambio, compiten por hacer `popTop()`, allí implementamos el top *lock-free* (usando `CAS`).
- En el caso de estar vacía la deque de la víctima, `CAS` puede traer `null`.
- Cuando queda un solo elemento en la deque, `popBottom` también usa un mecanismo basado en `CAS`, ahí sí el thread dueño está compitiendo con los ladrones.
- El libro muestra implementaciones de capacidad acotada y capacidad no acotada de la estructura, mostrando su lineazabilidad.

work dealing

- En vez de robar, los threads **rebalancean** periódicamente su carga con otro elegido al azar:
- Para decidir hacerlo, lanza una **moneda cargada**: rebalancear con probabilidad $1/(s+1)$, inversamente proporcional al tamaño s de su deque;
- Elige un socio al azar, toma **locks en las dos colas** (en orden de thread ID, para no tener deadlock) y, si la diferencia de cargas supera un umbral, **transfiere tareas hasta igualarlas**.
- **Ventajas:** cada intercambio mueve *muchas* tareas (con stealing, si un thread concentra el trabajo, los ladrones se llevan *una* tarea por vez, compitiendo por su deque); y con tareas parejas garantiza *fairness*: la carga esperada de cada deque queda cerca del promedio.

Paralelismo de datos

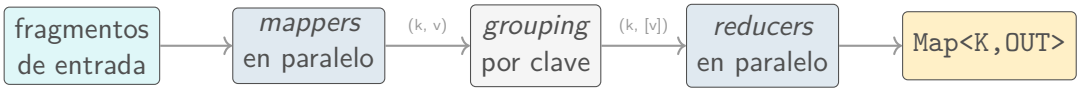
Paralelismo de datos

- Tenemos una misma tarea, a realizar en un conjunto grande de datos.
- Repartimos los datos, múltuples threads ejecutan *el mismo código* sobre datos distintos.
- No es igual a *repartir distintas tareas* entre los threads.
- Dos técnicas: *MapReduce* para procesamiento genérico, y *streams*, una solución de más alto nivel implementada en algunos lenguajes (por ejemplo Java).

El patrón *MapReduce*

1. Se divide la tarea en *fragmentos* que pueden ser procesados independientemente.
2. Los fragmentos son convertidos en n tareas *mapper*. Cada tarea *mapper* convierte un fragmento de la entrada en pares `(clave, valor)` (la semántica de la clave y el valor la da la aplicación).
3. Se hace el *grouping* de los pares por clave: para cada clave k, se arma la lista de sus valores en todas sus apariciones en los fragmentos mapeados.
4. Cada lista asociada a una clave se vuelve una tarea *reducer*: para cada clave k, se computa un único valor de salida $\texttt{output}_k$ basado en la lista de valores asociados a k.
5. Se genera el mapa de salida $(\texttt{clave}, \texttt{output}_{\texttt{clave}})$

MapReduce, esquema



- El framework coordina las fases
- los mappers y reducers son tareas de un pool *fork-join*
- el usuario solo necesita escribir dos funciones.

Clase abstracta Mapper

```
public abstract class Mapper<IN, K, V> extends RecursiveTask<Map<K, V>> {
  protected IN input;

  public void setInput(IN anInput) {
    input = anInput;
  }

  // (heredado de RecursiveTask)
  // compute(): para cada elemento del input,
  // producir pares (clave, valor)

}
```

Clase abstracta Reducer

```
public abstract class Reducer<K, V, OUT> extends RecursiveTask<OUT> {
  protected K key;
  protected List<V> valueList;

  public void setInput(K aKey, List<V> aList) {
    key = aKey;
    valueList = aList;
  }

  // (heredado de RecursiveTask)
  // compute(): para el key, producir
  // el OUT

}
```

El framework: la interfaz

```
class MapReduce<IN, K, V, OUT> implements Callable<Map<K, OUT>> {
  MapReduce()
  Map<K,OUT> call()
  void setMapperSupplier(Supplier<Mapper<IN,K,V>> aMapperSupplier)
  void setReducerSupplier(Supplier<Reducer<K,V,OUT>> aReducerSupplier)
  void setInput(List<IN> anInput)
}
```

- El `MapperSupplier` y `ReducerSupplier` se setean con funciones anónimas, luego el framework crea los mappers y reducers que el usuario pida.
- El método `call()` corre el pipeline completo: crea y forkea los mappers, hace el grouping, crear y forkea los reducers, y devuelve el resultado.

Implementación del framework

```
public Map<K, OUT> call() {
  Set<Mapper<IN, K, V>> mappers = new HashSet<>();
  for (IN input : inputList) {          paso 2: mappers
    Mapper<IN, K, V> mapper = mapperSupplier.get();
    mapper.setInput(input);
    pool.execute(mapper);
    mappers.add(mapper);
  }
  Map<K, List<V>> mapResults = new HashMap<>();
  for (Mapper<IN, K, V> mapper : mappers) {
    Map<K, V> map = mapper.join();      paso 3: grouping
    for (K key : map.keySet()) {
      mapResults.putIfAbsent(key, new LinkedList<>());
      mapResults.get(key).add(map.get(key));
    }
  }
                              //sigue
```

Implementación del framework (cont.)

```
    Map<K, Reducer<K, V, OUT>> reducers = new HashMap<>();
    mapResults.forEach((k, v) -> {      paso 4: reducers
      Reducer<K, V, OUT> reducer = reducerSupplier.get();
      reducer.setInput(k, v);
      pool.execute(reducer);
      reducers.put(k, reducer);
    });
    Map<K, OUT> result = new HashMap<>();
    reducers.forEach((k, r) ->
        result.put(k, r.join()));       paso 5: mapa salida
    return result;
}
```

Todo el paralelismo lo provee el `ForkJoinPool` (con work stealing) y las garantías de memoria de `join()`.

Ejemplo 1: WordCount

Como primer ejemplo, realizamos la siguiente tarea sencilla:

- Contamos con un corpus de texto que consiste en una lista de palabras
- El objetivo es contar las apariciones de cada palabra
- Algorítmicamente es sencillo, nos va a servir para tener un ejemplo inicial del uso del framework.

Ejemplo 1: WordCount - Código

```
public class WordCount {
  static List<String> text;
  static int numWorkers = ...;

  public static void main(String[] args) {
    text = readFile(...);
    List<List<String>> inputs =
        splitInputs(text, numWorkers);
    MapReduce<List<String>, String, Long, Long>
        mapReduce = new MapReduce<>();
    mapReduce.setMapperSupplier(
        WordCount.WCMapper::new);
    mapReduce.setReducerSupplier(
        WordCount.WCReducer::new);
    mapReduce.setInput(inputs);
    displayOutput(mapReduce.call());
  }
}
```

Ejemplo 1: WordCount: Mapper y Reducer

```
static class WCMapper
    extends Mapper<List<String>, String, Long> {
  public Map<String, Long> compute() {
    Map<String, Long> map = new HashMap<>();
    for (String word : input)
      map.merge(word, 1L, (x, y) -> x + y);
    return map;
  }
}
static class WCReducer
    extends Reducer<String, Long, Long> {
  public Long compute() {
    long count = 0;
    for (long c : valueList) count += c;
    return count;
  }
}
```

Ejemplo 2: *KMeans* - Introducción

- Idea: Nos interesa clasificar múltiples textos a lo que sospechamos son k autores distintos.
- Para esta clasificación, vamos a usar una métrica basada en la frecuencia de N palabras previamente elegidas en el texto.
- Vemos al vector de las frecuencias de esas N palabras (normalizadas para sumar 1) como un punto que podemos representar en el espacio N-dimensional
- La *distancia* entre dos de estos vectores es la distancia euclidiana de sus puntos en ese espacio.
- La idea es observar la manera en que los puntos se agrupan en el espacio para deducir su procedencia.
- Para este ejemplo, vamos a tomar como input los vectores ya calculados.

Ejemplo 2: *KMeans* - Iteraciones

- Fijamos (en principio, arbitrariamente) puntos en el espacio que van a ser *centros*, y representarán a cada potencial autor. Una posibilidad es tomar el valor de puntos en el set, elegidos aleatoriamente (método de Forgy).
- Para cada texto, se lo asocia al centro que esté más cerca en el espacio, cada cúmulo de vectores con mismo centro recibe el nombre *cluster*.
- Una vez asociados todos los puntos, se *itera nuevamente*, cambiando los centros originales por el baricentro (promedio) de cada cluster, hasta que el centro entre una iteración y la siguiente disten menos de un ε paramétrico.

Ejemplo 2: *KMeans* - Esquema *MapReduce*

- **Mapper**: fragmento de puntos $\rightarrow$ pares (`id de centro más cercano, punto`).
- **Reducer**: para cada cluster, computar su nuevo centro (el baricentro de los puntos asignados).

Ejemplo 2: *KMeans* - Código

```
public class KMeans {
  static List<Point> points;
  static Map<Integer, Point> centers;

  static final double EPSILON = 0.01;

  public static void main(String[] args) {
    points = readFile("cluster.dat");
    centers = Point.randomDistinctCenters(points);
    MapReduce<List<Point>, Integer, List<Point>, Point>
        mapReduce = new MapReduce<>();
    mapReduce.setMapperSupplier(KMeans.KMMapper::new);
    mapReduce.setReducerSupplier(KMeans.KMReducer::new);
    mapReduce.setInput(splitInput(points, numWorkerThreads));
    double convergence = 1.0;
    while (convergence > EPSILON) {
      Map<Integer, Point> newCenters = mapReduce.call();
      convergence = distance(centers, newCenters);
      centers = newCenters;
    }
  }
}
```

Ejemplo 2: *KMeans* - Código (cont.)

```
static class KMMapper
    extends Mapper<List<Point>, Integer, List<Point>> {
  public Map<Integer, List<Point>> compute() {
    Map<Integer, List<Point>> map = new HashMap<>();
    for (Point point : input) {
      int myCenter = closestCenter(centers, point);
      map.putIfAbsent(myCenter, new LinkedList<>());
      map.get(myCenter).add(point);
    }
    return map;
  }
}
static class KMReducer
    extends Reducer<Integer, List<Point>, Point> {
  public Point compute() {
    List<Point> cluster = new LinkedList<>();
    for (List<Point> list : valueList)
      cluster.addAll(list);
    return Point.barycenter(cluster);
  }
}
```

Streams: paralelismo de datos en el lenguaje

- En varios lenguajes, por ejemplo Java, existen los *streams*.
- Un stream es un *pipeline* que lleva valores desde una *fuente* (una `List`, un archivo, etc.) a través de *transformaciones* hacia un destino. La fuente puede ser no acotada (ej. un generador de números)
- Las transformaciones se comportan de manera funcional (no mutan valores, devuelven nuevos). Como no hay estado mutable compartido, no hay condición de carrera.
- Las transformaciones intermedias (`map`, `filter`, `flatMap`) son *lazy*: no computan nada hasta que una operación *terminal* (`collect`, `reduce`, `forEach`) lo exige; este *laziness* permite optimizar: el runtime fusiona transformaciones en un solo recorrido.
- La operación terminal consume el stream, no se puede reutilizar.

WordCount con streams

Podemos resolver el problema de *WordCount* generando un stream de palabras, donde la trasformación que hacemos es contar y agregar en el diccionario.

WordCount con streams - Código

```
public class WordCount {
  public static void main(String[] args) {
    List<String> text = readFile(...);
    Map<String, Long> map = text
      .stream()
      .collect(Collectors.groupingBy(
          Function.identity(),
          Collectors.counting()));
    displayOutput(map);
  }
}
```

Donde

- `readFile()` lee el archivo y lo deja separado en líneas (próximo slide)
- `groupingBy(k, counting())`: agrupa por la palabra (la clave es la palabra misma) y cuenta los elementos de cada grupo.

WordCount con streams - Código lectura

```
static List<String> readFile(String fileName) {
    Pattern pattern = Pattern.compile("\\P{L}+"); //toma acentos
    try (BufferedReader reader =                  //try-with-resource
            new BufferedReader(new FileReader(fileName))) {
        return reader
                .lines()
                .map(String::toLowerCase)
                .flatMap(s -> pattern.splitAsStream(s))
                .filter(w -> !w.isEmpty())        //filtra palabra vacía
                .collect(Collectors.toList());
    } catch (IOException ex) {
        throw new RuntimeException("Archivo no encontrado", ex);
    }
}
```

- `lines()`: el archivo como stream de líneas.
- `flatMap(s -> pattern.splitAsStream(s))`: de cada línea, sus palabras; `map(String::toLowerCase)`: cada palabra en minúsculas.

Streams paralelos

- `list.parallelStream()`: el runtime parte el stream en substreams, aplica las operaciones *en paralelo* y combina los resultados usando `ForkJoinPool`.
- Si queremos un control fino de *cómo* se hace la partición debemos usar herramientas adicionales. Java provee una clase `Spliterator<T>` con métodos para elegir el split. Si no, decide en función de la estructura de datos y CPUs disponibles.
- Paralelizar tiene un costo asociado. Para pocos elementos, el stream secuencial puede ganar.
- Si las transformaciones no son asociativas/conmutativas, el resultado puede cambiar entre corridas. Y el orden de `forEach` cambia:

```
Arrays.asList("a", "b", "c")
    .parallelStream()
    .forEach(s -> System.out.println(s));
```

Uso de funciones anónimas en streams

- Una lambda es *interferente* si altera la fuente del stream (normalmente lanza `ConcurrentModificationException`).

```
list.stream().forEach(s -> list.add(0));
```

- Una lambda es *stateful* cuando su efecto depende del estado del entorno: en paralelo, `add()` sobre una `List` no es *thread-safe*; hay potenciales condiciones de carrera.

```
source.stream().forEach(s -> target.add(s));
source.parallelStream().forEach(s -> target.add(s));
```

- En un stream, las lambdas que se usen deben tener transparencia referencial.

KMeans usando streams

Para *KMeans*, mantenemos las ideas que usábamos e incorporamos streams:

- Obtenemos el primer conjunto de centros (Forgy)
- Usamos un stream para calcular el grupo de cada punto (usando `GroupingByConcurrent`, la variante de `GroupBy` que usa un `ConcurrentMap`)
- Usamos un segundo stream para calcular el baricentro de cada cluster (los nuevos centros)
- Iteramos hasta que los nuevos centros están suficientemente cerca de los centros del paso anterior.

KMeans usando streams - Cálculo de baricentro

```
static Point barycenterStream(List<Point> cluster) {
  double numPoints = cluster.size();
  Optional<Point> sum = cluster
      .parallelStream()
      .reduce(Point::plus);    //suma
  return sum.get().scale(1 / numPoints);
}
```

KMeans usando streams - iteración

```
Map<Integer, List<Point>> clusters =
    points.parallelStream()
          .collect(Collectors.groupingByConcurrent(
            p -> closestCenter(centers, p)));
Map<Integer, Point> newCenters =
    clusters.entrySet().parallelStream()
            .collect(Collectors.toConcurrentMap(
                e -> e.getKey(),
                e -> Point.barycenterStream(
                        e.getValue())));
```

Bibliografía

- Herlihy, Shavit, Luchangco y Spear, *The Art of Multiprocessor Programming*, 2da ed., capítulos 16 y 17.
- Lea, D., *A Java Fork/Join Framework*, 2000. `http://doi.org/10.1145/337449.337465`
- Cormen, Leiserson, Rivest y Stein, *Introduction to Algorithms*, Capítulo 27 (3ra ed.), Capítulo 26 (4ta ed.)